

Objectives

- Strengthen the use of Python functions and variable arguments (*args, **kwargs).
- Practice dictionary manipulation using built-in methods.
- Apply Object-Oriented Programming concepts, including encapsulation, properties, inheritance, multiple inheritance, and polymorphism.
- Develop clean, structured, and reusable Python code.
- Improve problem-solving and software design skills using Python.

Submission Instructions

The submission must be a **written report** that includes:

- A clear presentation of each exercise.
- The Python source code for each solution.
- Explanations of the design choices and implementation.
- Screenshots showing the execution and results of your programs.

The report must be submitted on **Google Classroom** in the course: **Advanced Python Programming Assignments**

Excellence Master's Degree - ADIA / ISSE

Academic Year

2025/2026

Exercise 1

Write the following functions:

1. `is_even(n)`
Returns True if n is even, otherwise False.
2. `sum_list(values)`
Returns the sum of all elements in a list.
3. `max_list(values)`
Returns the maximum value in a list without using the built-in `max()` function.

Your implementation must handle invalid inputs appropriately.

Exercise 2

Implement a function:

`calc(op, *args)`

Requirements:

- `op` is a string specifying the operation: "sum", "prod", "min", "max".
 - `*args` represents a variable number of numeric arguments.
 - Implement the logic explicitly using loops.
 - Define a consistent behavior for unsupported operations and empty arguments.
-

Exercise 3

Implement a function:

`build_student_profile(**kwargs)`

This function returns a dictionary representing a student profile.

Requirements:

- The dictionary must support the following keys: name, age, major, grades.
- Provide default values for missing keys. ✍

- Add a derived key status based on the value of age.
 - Use appropriate dictionary techniques to build and manage the profile.
-

Exercise 4

You are given an inventory represented as a dictionary:

```
{ product_name: quantity }
```

Implement the following functions using dictionary methods such as `get`, `setdefault`, `update`, `pop`, `keys`, `values`, and `items`:

1. `add_item(inventory, name, quantity)`
Adds a quantity to an existing product or creates it if it does not exist.
 2. `remove_item(inventory, name, quantity)`
Removes a quantity only if sufficient stock exists.
 3. `merge_inventories(inventory, new_inventory)`
Merges two inventories by correctly combining quantities.
 4. `delete_product(inventory, name)`
Removes a product safely from the inventory.
-

Exercise 5

Create a class `Student` with the following requirements:

- Private attributes: `__name`, `__age`
 - Constructor with validation rules
 - Getter and setter methods using `@property`
 - Validation when modifying attributes
 - A method `info()` that returns a formatted string describing the student
-

Exercise 6

Create a class `BankAccount` with:

- Private attributes: `__owner`, `__balance`
- Methods:

- deposit(amount)
 - withdraw(amount)
 - transfer_to(other_account, amount)
 - Use @property to provide controlled access to attributes
 - Ensure the integrity of the account balance
-

Exercise 7

Create a base class Employee with:

- Private attributes
- A method monthly_pay()

Create derived classes that inherit from Employee and override the monthly_pay() method using their own logic.

Create a function that processes a list of employees and computes the total payroll using polymorphism.

Exercise 8

Create the following classes:

- A base class Notifier with a method send(message)
- A class EmailMixin with a method send_email(message)
- A class SMSMixin with a method send_sms(message)

Create a class SmartNotifier that inherits from multiple classes and provides a unified send(message, channel) method.

Create a function that accepts a list of notifier objects and sends a message to all of them using polymorphism.

Problematic — ADIA

Modern data analytics and AI systems must manage multiple types of machine learning models, each with different characteristics such as training time, accuracy, and prediction behavior. These models must be organized in a flexible and extensible way to allow unified evaluation and comparison.

Design an object-oriented system to represent and manage different types of AI models.

Requirements:

- Create a base class `Model` with private attributes such as:
 - `model name`
 - `training time`
 - `accuracy`
- Use `@property` to provide controlled access to these attributes.
- Implement derived classes such as:
 - `ClassificationModel`
 - `RegressionModel`
- Each derived class must implement its own version of a method:

`predict(data)`

- Implement a method `evaluate()` that returns performance information.
- Create a manager class `ModelManager` that can store multiple models and evaluate them using polymorphism.

The system must demonstrate:

- encapsulation,
- inheritance,
- properties,
- and polymorphism.

Problematic — IISE

Embedded systems often interact with multiple sensors such as temperature sensors, pressure sensors, and motion sensors. Each sensor has specific characteristics but must be managed through a unified interface.

Design an object-oriented system to represent and manage sensors.

Requirements:

- Create a base class `Sensor` with private attributes such as:
 - sensor ID
 - sensor name
 - current value
- Use `@property` to control access to these attributes.
- Implement derived classes such as:
 - `TemperatureSensor`
 - `PressureSensor`
 - `MotionSensor`
- Each sensor must implement its own version of a method:

`read_value()`

- Create a class `SensorSystem` that manages multiple sensors and reads values from all sensors using polymorphism.
- Add a mixin class `LoggerMixin` that allows sensors to log their readings.

The system must demonstrate:

- encapsulation,
- inheritance,
- multiple inheritance,
- properties,
- and polymorphism.